kerneltests: Verified Aggregated Kernel Hypothesis Tests in Python

Ankith Burki

Abstract

Aggregated kernel tests—MMDAgg, MMD-FUSE, KSDAgg and their linear-time incomplete U-statistic variants—adapt to unknown smoothness without data splitting and are among the most powerful nonparametric tests available for two-sample, independence and goodness-of-fit problems. Their implementations are spread across separate single-paper repositories with different interfaces and no automated tests. `kerneltests` unifies six such tests behind one API, verifies every port against the original reference code (identical decisions, and statistics within relative error 10^{-10}, in all 302 configurations compared), fixes five defects found in the references, and provides an algebraically cheaper wild bootstrap and an optional JAX backend that together run MMDAgg 2.8–4.3× faster than the reference on CPU. The library is MIT-licensed, has 141 tests at 98.9% line coverage, and is documented at [TODO: URL].

1 Introduction

Kernel hypothesis tests compare distributions through embeddings in a reproducing kernel Hilbert space: the maximum mean discrepancy (MMD) for two-sample testing, the Hilbert–Schmidt independence criterion (HSIC) for independence, and the kernel Stein discrepancy (KSD) for goodness of fit. Their power depends strongly on the kernel and bandwidth. Aggregated tests (Schrab et al., 2023, 2022a) combine many kernels and bandwidths with a level correction and adapt to the unknown smoothness of the alternative without splitting the data; MMD-FUSE (Biggs et al., 2023) fuses them through a soft maximum; incomplete U-statistics (Schrab et al., 2022b) reduce the cost from quadratic to linear in the sample size.

[TODO: one or two sentences on applications, with citations, e.g. distribution-shift detection and evaluation of generative models.] The reference implementations are research code: one repository per paper, no automated tests, input checks by `assert`, and different interfaces. `kerneltests` is a single maintained library for practitioners, and a trustworthy baseline for researchers.

2 Design

The public API has one function per testing problem. Each returns a `TestResult` carrying the decision, the level and per-kernel diagnostics (statistics, quantiles, p-values and thresholds):

```
import kt
kt.two_sample(X, Y)                 # MMDAgg; also method="mmdfuse", "mmdagginc"
kt.independence(X, Y)               # HSICAggInc
kt.goodness_of_fit(X, score_X)      # KSDAgg; also method="ksdagginc"
```

Low-level functions expose every option of the original methods (for MMDAgg: 19 kernel configurations, four weighting schemes, wild bootstrap or permutations). Bandwidth selection, weights and kernels are shared across tests. The core depends only on NumPy and SciPy; JAX is an optional backend.

3 Verification

Agreement with the published methods. Each NumPy port reproduces its reference's random-number protocol, so for a fixed seed the decision, statistics, quantiles and thresholds can be compared directly. In every configuration compared, decisions and p-values are identical and statistics agree to relative error 10^{-10}; configurations compared: MMDAgg in 152 (every kernel option, weighting, odd and even bandwidth counts, both resampling schemes), KSDAgg in 30, and the incomplete U-statistic tests in 80. MMD-FUSE's reference uses JAX's random number generator, so its permutations were drawn by the reference and passed to the port; the 40 configurations agree to a relative error of 1.5×10^{-11}. The JAX backend agrees with the NumPy path in all 152 MMDAgg configurations.

Level. Over 30 settings (five tests; $d \in \{1, 5, 10\}$; $n \in \{100, 500\}$; 1000 repetitions each, $\alpha = 0.05$), no 95% Clopper–Pearson interval for the rejection rate under the null lies above α (range 0.002–0.060). [TODO: at $n = 500$ these runs used $B_1 = B_2 = 100$, which makes MMD and HSIC conservative; replace with the $B = 500$ rerun.]

Power. At $n = 500$ ($B_1 = B_2 = 500$, 200 repetitions), power rises from the nominal level to near one as the effect grows for every test: for a mean shift of 0.3 in $d = 5$, MMDAgg rejects in 89% of runs and MMDAggInc in 83%; for a 0.3 shift under the KSD model, both reach 100%; HSICAggInc detects linear dependence of strength 0.3 in 96% of runs. As expected, the incomplete U-statistic variants trade a little power for linear cost (for example KSD scale change 0.3: 0.88 against 0.76).

Defects found. Exact comparison exposed five defects in the reference implementations, all fixed here with regression tests and reported upstream [TODO: issue links]: (i) MMDAgg reports the Laplace-kernel p-values in place of the Gaussian ones under its default kernel (decisions unaffected); (ii) KSDAgg compares two equivalent rejection rules with an exact-equality assertion, so rounding makes it raise an error on some ordinary inputs; (iii) AggInc uses the `warnings` module without importing it; (iv) AggInc's HSIC input check validates the first bandwidth list twice; (v) AggInc's automatic batch size allocates all bootstrap samples at once (about 9 GB at $n = 2000$, killing the process).

4 Performance

For equal sample sizes the wild-bootstrap statistic is $r^\top K r$ with $r = (\epsilon, -\epsilon)$ and $K \in \mathbb{R}^{2n \times 2n}$; this equals $\epsilon^\top H \epsilon$ with $H = K_{XX} + K_{YY} - K_{XY} - K_{YX} \in \mathbb{R}^{n \times n}$, a fourfold reduction in matrix-product cost. The optional JAX backend additionally compiles kernel construction and batches all bandwidths.

n	reference (s)	kt NumPy (s)	kt JAX (s)	JAX vs reference
500	3.52	2.28	0.82	4.3×
1000	10.28	5.90	3.71	2.8×
2000	40.38	16.51	9.78	4.1×
4000	100.0	33.08	30.63	3.3×

MMDAgg, $d = 10$, $B_1 = B_2 = 500$, median of three runs after a warm-up, two-core Intel Xeon virtual machine. Decisions agreed at every n. GPU execution is supported by the JAX backend but was not benchmarked.

5 Comparison with existing software

	kerneltests	hyppo 0.5.2	PyRKHSstats	torch-two-sample	reference packages
Problems (2S/Ind/GoF)	all three	all three	2S, Ind	2S	one each
Aggregated bandwidths	yes	no	no	no	yes
Linear-time variants	yes	no	no	no	AggInc only
Automated tests / CI	yes / yes	yes / yes	partial / no	yes / legacy	no / no
On PyPI	[TODO: yes]	yes	yes	no	no
License	MIT	MIT	GPL-3	BSD-3	MIT

hyppo (Panda et al., 2024) is the closest maintained library. With its defaults it uses a χ^2 approximation instead of resampling, which makes it 4–50× faster in our timings. Adaptivity is what the aggregated tests buy. On a perturbed-uniform alternative with fine-scale structure ($n = 200$, 100 repetitions), `kerneltests` rejected in 100% of runs against 20% (hyppo MMD) and 29% (energy distance). On a plain mean shift, hyppo was more powerful (0.73 against 0.58–0.61). [TODO: replace with the $B = 500$ power study and stronger independence alternatives.] The two libraries are complementary: hyppo for fast screening, `kerneltests` when power against unknown or fine-scale differences matters.

6 Community and availability

Source: [TODO: GitHub URL]; documentation: [TODO: URL]; PyPI: [TODO: package]. Continuous integration covers Linux, macOS and Windows with Python 3.9–3.13. The repository includes contribution guidelines that require every new port to demonstrate parity with its reference. [TODO: user base evidence: downloads, dependents, external contributors, user issues.]

Acknowledgments

[TODO: e.g. thanks to the authors of the reference implementations.]

References

F. Biggs, A. Schrab, A. Gretton. MMD-FUSE: learning and combining kernels for two-sample testing without data splitting. *NeurIPS*, 2023.

S. Panda, S. Palaniappan, J. Xiong, et al. hyppo: a multivariate hypothesis testing Python package. [TODO: verify venue and year].

A. Schrab, I. Kim, M. Albert, B. Laurent, B. Guedj, A. Gretton. MMD aggregated two-sample test. *JMLR* 24(194):1–81, 2023.

A. Schrab, B. Guedj, A. Gretton. KSD aggregated goodness-of-fit test. *NeurIPS*, 2022.

A. Schrab, I. Kim, B. Guedj, A. Gretton. Efficient aggregated kernel tests using incomplete U-statistics. *NeurIPS*, 2022.